

pharo-agentic-browser の紹介

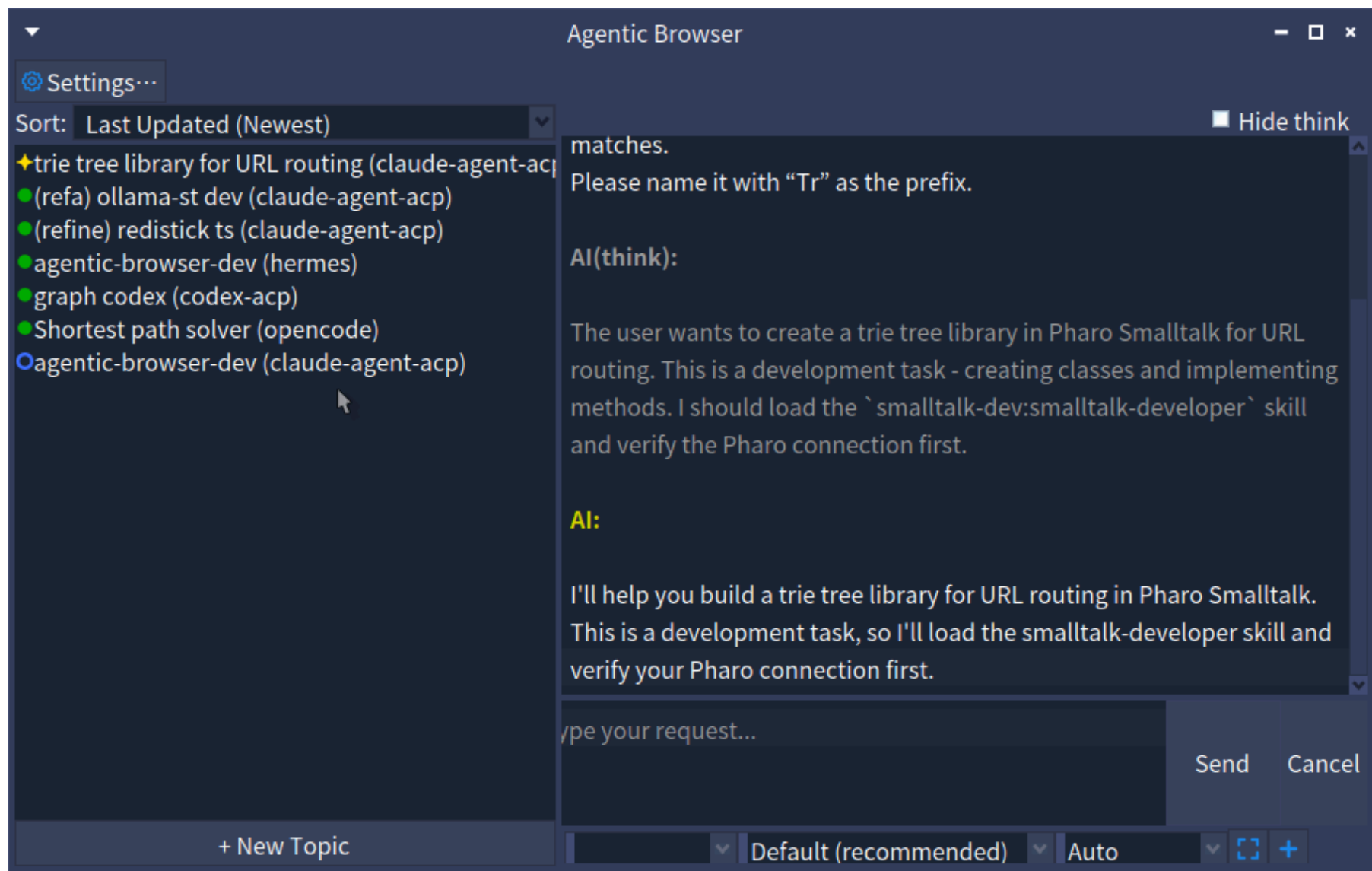
Pharoの統合AIコーディング環境

梅澤 真史

<https://github.com/mumez/pharo-agentic-browser>

pharo-agentic-browser とは？

Claude Code、Codex、OpenCode など、複数のAI コーディングエージェントを
Pharo から利用できる**統合AIコーディングツール**です。



各AIとのセッションは **トピック** として管理します:

1. トピックを作成し、ACP 対応エージェントを選択
2. リクエストを入力 + `@ClassName` でコードを参照、スクリーンショットの添付も可
3. AI が自律的に作業（タスク分解、コード変更、テスト）
4. AI が承認を必要とする場合、チャット内で一時停止して確認
5. プルダウンで応答すると AI が再開
6. トピックの状態は常にサイドバーで確認可能

開発の動機

AI コーディングエージェント専用の GUI ツールが標準になりつつあります:

- **Claude Desktop** — ツール、MCP、ファイルアクセスを備えた GUI版Claude
- **Codex App** — OpenAI の自律的なコーディング環境
- **Cursor / Antigravity / Kiro** — AI ネイティブなエディタ

これらのツールは、AI エージェントとやり取りする際の敷居を下げ、単純なチャットを超えたものになっています。

時代	パラダイム	例
黎明期	エディタサイドバーでのチャット	ChatGPT, GitHub Copilot
少し前	エージェントモードを持つ AI ファーストな IDE	Cursor, Windsurf
現在	タスク全体をエージェントに委譲	Antigravity 2.0, Codex App

AI は、単なるコード行の補完ではなく、**機能単位のタスク**を扱えるようになりました。

ツールは進化しています。UI はもはや「エディタ + チャット」ではなく、**セッションのオーケストレーション**へと向かっています。

なぜ Pharo にもこれが必要か

Pharo 開発者も同じパラダイムを享受すべき:

- リッチなライブ環境 — クラス、メソッド、ランタイムがすぐそこにある
- **pharo-acp** が複数エージェント向けの ACP クライアントをすでに提供
- 複数プロジェクトを1つのイメージ内で並行実行可能

複数の AI エージェントを制御できる Pharo ネイティブなツールが、次のステップとして自然です。

優位性	詳細
コンテキストを直接渡せる	<code>@ClassName</code> 、 <code>@Class>>method</code> 、スクリーンショット — コピー不要
エージェント非依存	pharo-acp 経由でさまざまなエージェントと連携
Pharo との統合	テスト、コードの変更監視がライブイメージの中で利用可能
並行セッション	複数エージェントを異なるトピックで同時実行

インストール

Pharo 12+ のイメージで Playground を開き、以下を評価:

```
Metacello new  
  baseline: 'AgenticBrowser';  
  repository: 'github://mumez/pharo-agentic-browser:main/src';  
  load: 'all'.
```

AgenticBrowserを開く:

```
AgenticBrowser open.
```

ACP 対応であれば、どのエージェントも利用可能 - プリセット一覧:

エージェント	インストール
Claude Code	<code>npm install -g @agentclientprotocol/claude-agent-acp</code>
Codex	<code>npm install -g @agentclientprotocol/codex-acp</code>
Gemini CLI	ACP 組み込み (<code>gemini --acp</code>)
Copilot CLI	ACP 組み込み (<code>copilot --acp --stdio</code>)
Cursor CLI	ACP 組み込み (<code>agent acp</code>)

OpenCode、Kilo Code、Kiro CLI などにも対応

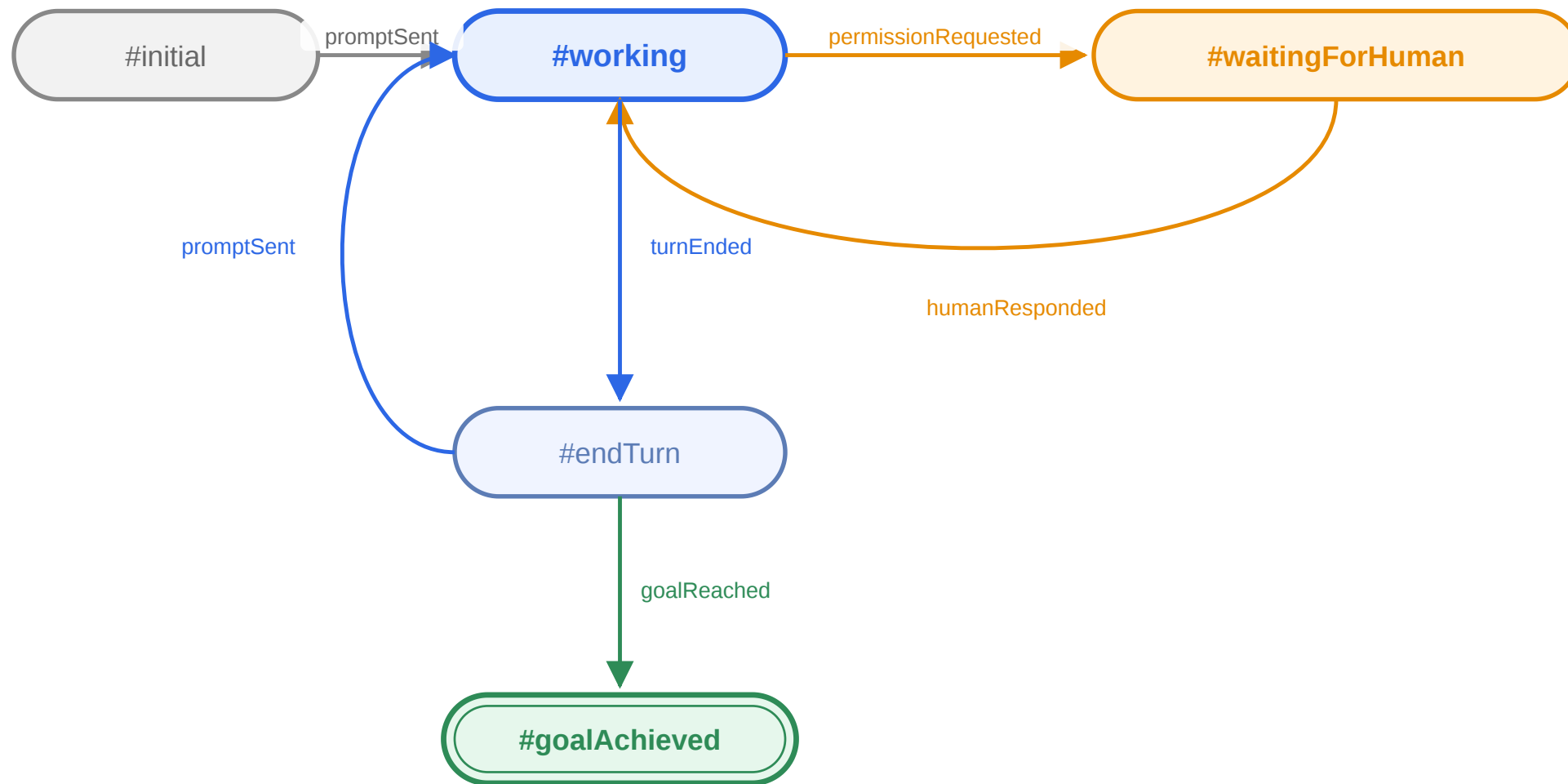
推奨: Smalltalk コードの質を上げるため、エージェントに [smalltalk-dev-plugin](#) をインストールしておく

基本機能

1. + **New Topic** をクリック
2. タイトルを入力し、コーディングエージェントを選択
3. (任意) 既存のプロジェクトディレクトリを設定
4. **Create** をクリック — トピックが左サイドバーに表示される
5. (任意) 右クリック → **Set Target Packages...** で追跡対象パッケージを設定

最初のメッセージには、Smalltalk開発者スキルを有効化するため、自動的に `/st-buddy` が付与されます。

各トピックの状態はステートマシンで明確に管理:



アイコン	状態	意味
	working	AI が作業中
	waitingForHuman	AI が承認を待っている
	endTurn	ターン完了
	goalAchieved	ゴール達成

AI が承認を要求すると:

- ステータスが **?** (waitingForHuman) に変化
- **Send** ボタンが **Allow** に変化
- **Cancel** ボタンが **Deny** に変化

ボタンをクリックして応答すると、AI がシームレスに再開します。

モーダルダイアログはありません。承認は会話フローの一部です。

チャット内で Pharo のクラスやメソッドを直接参照できます:

```
@QueryClass @DBAdapter>>connect please refactor this
```

AgenticBrowser は各 `@mention` を Tonel のソースとして解決し、ACP のテキストリソースにして添付します — コピペ不要。

ドラッグ&ドロップ

- System Browser から クラス をドラッグ → `@ClassName` を挿入
- System Browser から メソッド をドラッグ → `@ClassName>>methodName` を挿入

【 】 ボタンをクリックしてスクリーンショットを添付:

1. ボタンをクリック — カーソルが十字カーソルに変化
2. ドラッグして範囲を選択
3. @sc-20260528-001.png のようなメンションが入力欄に挿入される
4. 送信 — PNG が画像リソースとしてプロンプトに添付される

ファイルは <agenticBrowserRoot>/screenshots/sc-YYYYMMDD-NNN.png に保存されます。

+ ボタンをクリックしてディスク上の任意のファイルを添付:

1. ボタンをクリック — ファイル選択ダイアログが開く
2. ファイルを選択 — `[filename]` のようなメンションが入力欄に挿入される
3. 送信 — ファイルの内容がテキストリソースとして添付される

サイズの大きいファイルは、プロンプトに埋め込まれる前に `maxAttachmentSize` (デフォルト 5MB) に切り詰められます。

送信前に `[filename]` のメンションテキストを削除すると、添付はキャンセルされます。

トピックを右クリック → **Set Goal...** で完了条件を入力:

```
all tests pass
```

AgenticBrowser は AI にゴール用のプロンプトを送信します:

Goal has been set: all tests pass. When the goal is achieved, summarize and report in result-<topic-id>.md. Keep retrying until the goal is achieved.

`result-<topic-id>.md` が作成されると、トピックは ☒ (`#goalAchieved`) に遷移します。

ゴール達成時に2つのフックが発火します:

アナウンサー

```
topic announcer
  when: AbTopicGoalAchieved
  do: [:ann | Transcript crShow: ann topic title , ' achieved: ' , ann goal result].
```

コールバックブロック

```
topic whenGoalAchieved: [:goal | Transcript crShow: goal result].
```

ゴール達成イベントを独自の自動化ワークフローに統合できます。

トピックは Pharo の **Fuel** シリアライザを使って `ab-topics.fuel` に自動保存され、イメージ再起動後も残ります。

手動での保存・復元:

```
AbTopicManager save.  
AbTopicManager load.
```

トピックごとの状態（設定、ステータス、会話）はすべて永続化されます。

トピックに関連するパッケージへの編集をイメージ内で監視:

- トピックを右クリック → **Set Target Packages...** で対象プレフィックスを設定 (例: `#('AgenticBrowser-*')`)
- 対象のクラス/メソッドが保存されると、チャットにシステムメッセージが表示され、確認後にパッケージがエクスポートされる
- **追跡対象外**のパッケージへの編集は追跡候補として収集され、**Apply Updated External Packages** で昇格可能

イメージ内でユーザ自身が変更した内容と、AI が見ている Tonel ソースとを同期させ続けます。

カスタマイズ

新規トピックの作業ディレクトリは、`<agenticBrowserRoot>/topic-template` をもとに生成されます:

- デフォルトでは `smalltalk-dev-plugin` 向けに調整された `CLAUDE.md` / `AGENTS.md` を同梱
- `.claude`、`.opencode` などのエージェント設定ディレクトリを配置できる — スキル、コマンド、ルールを全トピックで共有
- 独自にカスタマイズしたものに置き換え可能

新しいトピックごとにコーディングエージェントを再設定する手間を省けます。トピックが既存のプロジェクトディレクトリを指している場合は適用されません。

AgenticBrowser のルートディレクトリに `mcp.json` を配置:

```
{
  "mcpServers": {
    "my-server": {
      "command": "uvx",
      "args": ["my-server-package"],
      "env": {"API_KEY": "value"}
    }
  }
}
```

- 組み込みの `smalltalk-interop` と `smalltalk-validator` サーバーはデフォルトで自動マージされる
- `useDefaultMcpServers: false` を設定すると独自の `mcp.json` のみを使用

Playground から

```
AbSettings default codingAgents: (AbSettings default codingAgents copyWith:  
  {'name' -> 'my-agent'.  
  'command' -> #('my-agent' '--acp')} asDictionary).  
AbSettings save.
```

または **ab-settings.json** を直接編集

エージェントは次回起動時に **New Topic** ダイアログのプリセットとして表示されます。

キー	デフォルト	説明
<code>useDefaultMcpServers</code>	<code>true</code>	組み込みの Smalltalk MCP サーバーをマージ
<code>aiPermissionWaitTimeoutSeconds</code>	<code>1800</code>	人間の承認待ちタイムアウト
<code>aiPermissionTimeoutOption</code>	<code>#reject_once</code>	自動応答: <code>allow_once</code> , <code>allow_always</code> , <code>reject_once</code>
<code>exportApprovalWaitTimeoutSeconds</code>	<code>30</code>	パッケージエクスポート承認のタイムアウト

設定は右クリック → **Edit Settings...** からトピックごとにも設定可能です。

Spec UI 以外にも、AgenticBrowser を利用する方法が2つあります:

- **Web UI** — WebSocketを使ったWebブラウザ用インターフェース。モバイル対応、トピックのライブ更新
→ [Web UI スライド](#)
- **Scripting API** — 複数エージェントのオーケストレーションを構築・実行するためのシンプルなDSL（逐次/並列ステップ、保存&読み込み）
→ [Scripting API スライド](#)

まとめ

pharo-agentic-browser は、コーディングエージェントへの委譲というパラダイムを Pharo にもたらしめます:

- **ネイティブ GUI** — イメージから離れずに複数の AI セッションを管理
- **エージェント非依存** — ACP 対応エージェントであればどれでも利用可能
- **リッチなコンテキスト** — コードメンション、ドラッグ&ドロップ、スクリーンキャプチャ
- **Human-in-the-Loop** — 会話の中で承認、割り込みダイアログなし
- **ゴール駆動** — 完了条件を設定し、フックも指定可能
- **拡張可能** — 最小限の設定でカスタム MCP サーバーやエージェントを追加

フィードバックとコントリビューションをお待ちしています！

<https://github.com/mumez/pharo-agentic-browser>